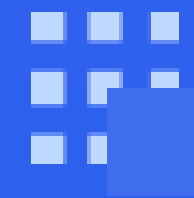




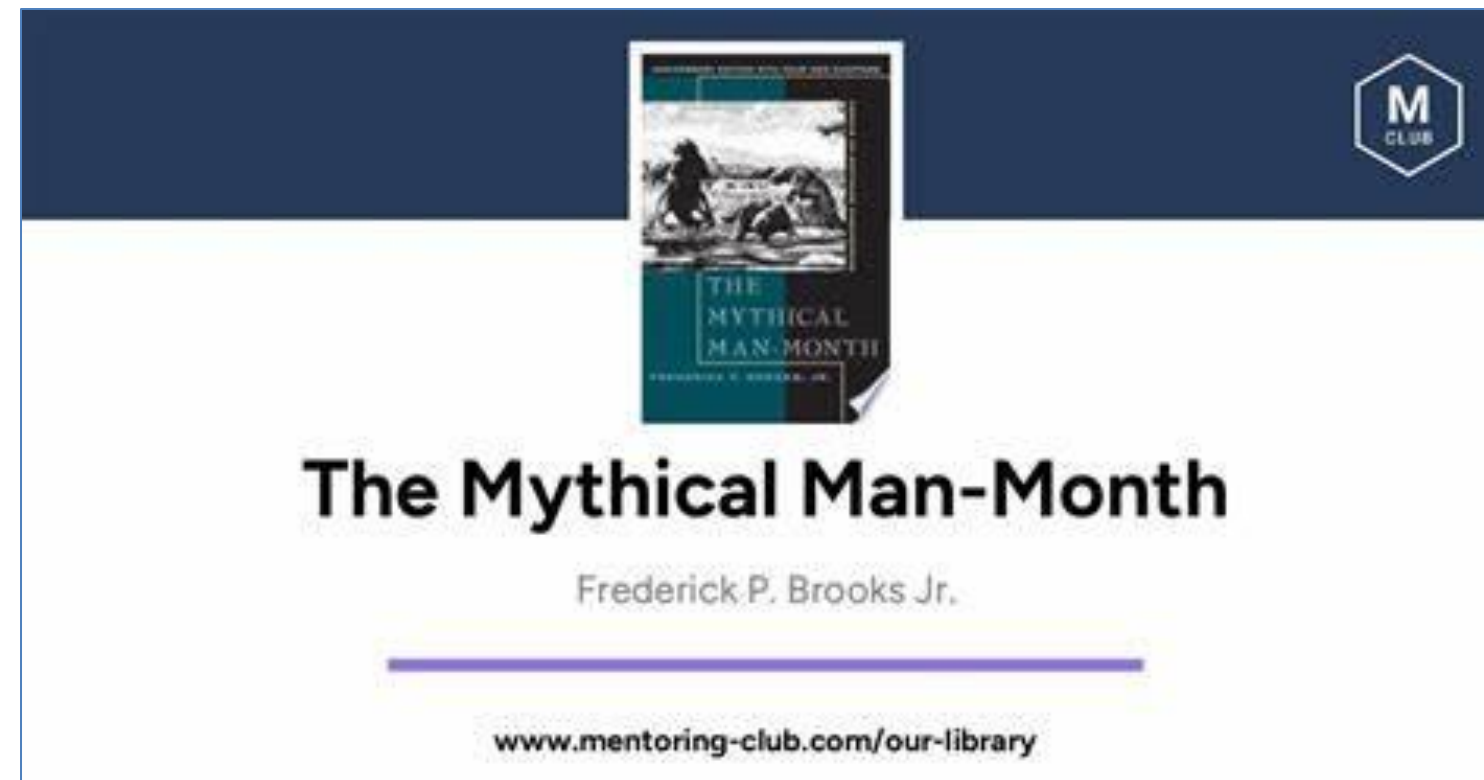
软件测试

Bug的非单调性



深入分析Bug ■ Bug的非单调性

- 单调性指函数值与自变量变化趋势一致。例如，随着自变量增大，函数值也增大。
- 非单调性指函数值变化与自变量变化不一致。例如，自变量增大时，函数值可能减小，当然也可能增大。
- 开发者经验增加，代码质量通常提高，这是单调性。系统复杂性与可维护性关系可能是非单调的，复杂性增加到一定程度，可维护性反而下降。



程序差异比较

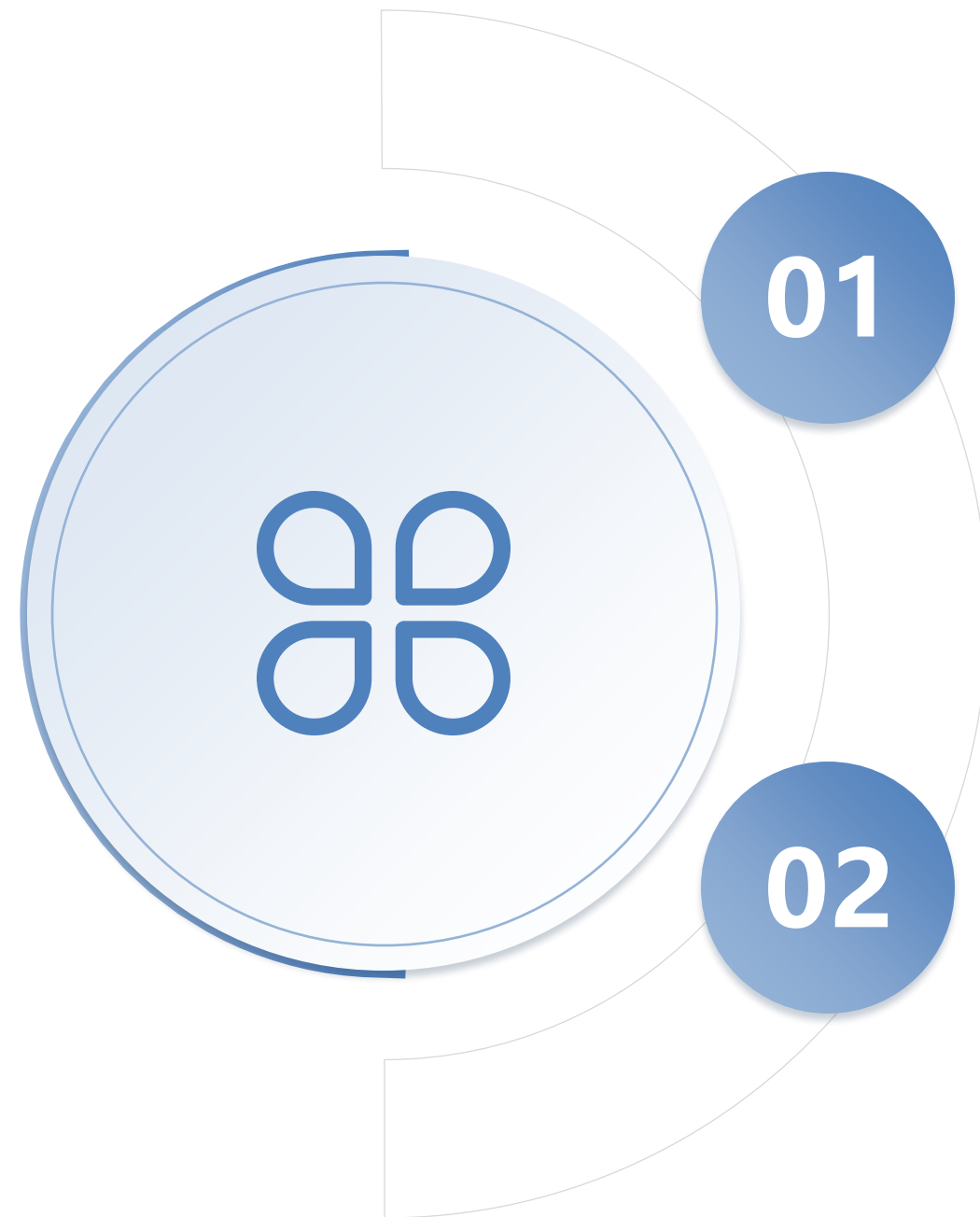
对于待测程序 P , P_1 和 P_2 分别为两个程序修改版本。
若 $P \setminus P_1 \subset P \setminus P_2$, 则称为 P_1 修改小于 P_2 , 记为 $P_1 <_P P_2$

正向的测试非单调性

P 为待测程序且 $P(t)$ 通过, P_1 和 P_2 为两个修改版本
且有 $P_1 <_P P_2$, 若 $P_1(t)$ 失效但 $P_2(t)$ 通过, 则称对于
程序 P 的两个修改版本 P_1 和 P_2 , 测试 t 是非单调的。

反向的测试非单调性

P 为待测程序且 $P(t)$ 失效, P_1 和 P_2 为两个修改版本
且有 $P_1 <_P P_2$, 若 $P_1(t)$ 通过但 $P_2(t)$ 失效, 则称对于
程序 P 的两个修改版本 P_1 和 P_2 , 测试 t 是非单调的。



```
void Mean1(double X[], int L) {  
    double mean, sum;  
    sum = 0;  
    for (int i = 1; i < L; i++) {  
        sum += X[i];  
    }  
    mean = sum / L;  
    printf("Mean: %f\n", mean);  
}
```



测试集

测试集 T:

t_1 输入 (3, 4, 5), 预期输出 4;

t_2 输入 (4, 3, 5), 预期输出 4。



测试结果

t_1 输入到MeanVar1 中, 输出3, t_1 失效。

t_2 输入到 MeanVar1中, 输出2.67, t_2 失效。

因此断定程序 Mean1 有 Bug。

事实上, for循环中应该i=1

```
void Mean2(double X[], int L) {  
    double mean, sum;  
    sum = 0;  
    for (int i = 1; i < L; i++) {  
        sum += X[i];  
    }  
    mean = sum / L-1;  
    printf("Mean: %f\n", mean);  
}
```



测试集

测试集 T:

t_1 输入 (3, 4, 5), 预期输出 4;

t_2 输入 (4, 3, 5), 预期输出 4。



测试和分析过程

- 将mean最后的除法分母修改为“L-1”，得到程序Mean2。
- 重新运行测试集，此时， t_1 失效， t_2 通过。
- 然而事实上，这个修复偏离正确方向更远了。

深入分析Bug Bug的非单调性

```
void Mean(double X[], int L) {  
    double mean, sum;  
    sum = 0;  
    for (int i = 0; i < L; i++) {  
        sum += X[i];  
    }  
    mean = sum / L;  
    printf("Mean: %f\n", mean);  
}
```

t_1 通过, t_2 通过

```
void Mean1(double X[], int L) {  
    double mean, sum;  
    sum = 0;  
    for (int i = 1; i < L; i++) {  
        sum += X[i];  
    }  
    mean = sum / L;  
    printf("Mean: %f\n", mean);  
}
```

t_1 失效, t_2 失效

```
void Mean2(double X[], int L) {  
    double mean, sum;  
    sum = 0;  
    for (int i = 1; i < L; i++) {  
        sum += X[i];  
    }  
    mean = sum / L-1;  
    printf("Mean: %f\n", mean);  
}
```

t_1 失效, t_2 通过

对比分析MeanVar, MeanVar1, MeanVar2三个程序的修复偏差, 以及测试集T的执行结果。

软件的复杂性常常带来Bug的非单调性

PART 01

代码层面的复杂性

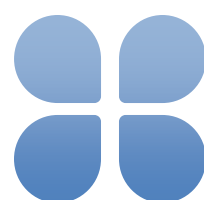
一个Fault可能涉及多行不连续代码，增加了修复的难度。修复时需考虑代码间的逻辑关联，避免引入新的问题。



PART 02

外部引用的复杂性

Bug可能涉及其他函数的外部引用，修复时需考虑整体架构。外部引用的修改可能引发连锁反应，导致新的问题出现。



PART 03

非单调性现象

在修复过程中，可能遇到“越修越坏”的情况，失效的测试数量增加。只有完整修复程序后，测试集才能全部通过，过程充满挑战。



深入分析Bug Bug的非单调性

Bug非单调性给程序员的调试和修复带来巨大挑战。

故障定位的困难

非单调性使得故障定位变得更加困难，需多次尝试和验证修复方案。开发者需具备丰富的经验和敏锐的洞察力，才能准确找到问题根源。

修复方案的不确定性

修复方案的不确定性增加，需谨慎处理，避免引入新的问题。每次修复后需重新评估，确保修复的正确性和对程序的稳定性。

实际案例分析

在复杂程序的测试和调试过程中，可能会遇到反向单调的情况。这些案例提醒开发者在修复过程中要保持警惕，避免陷入困境。

深入分析Bug ■ Bug的非单调性

```
void Mean(double X[], int L) {  
    double mean, sum;  
  
    sum = 0;  
  
    for (int i = 0; i < L; i++) {  
        sum += X[i];  
    }  
  
    mean = sum / L;  
  
    printf("Mean: %f\n", mean);  
}
```

```
void Mean1(double X[], int L) {  
    double mean, sum;  
  
    sum = 0;  
  
    for (int i = 1; i < L; i++) {  
        sum += X[i];  
    }  
  
    mean = sum / L;  
  
    printf("Mean: %f\n", mean);  
}
```

```
void Mean2(double X[], int L) {  
    double mean, sum;  
  
    sum = 0;  
  
    for (int i = 1; i < L; i++) {  
        sum += X[i];  
    }  
  
    mean = sum / L-1;  
  
    printf("Mean: %f\n", mean);  
}
```

测试集T: $t_1=[3,4,5]$, $t_2=[4,3,5]$, $t_3=[0,4,5]$, $t_4=[3,3,3]$

深入分析Bug ■ Bug的非单调性



测试设计

t_2 输入 (4, 3, 5), 预期输出 4
 t_3 输入 (0, 4, 5), 预期输出 3



测试执行

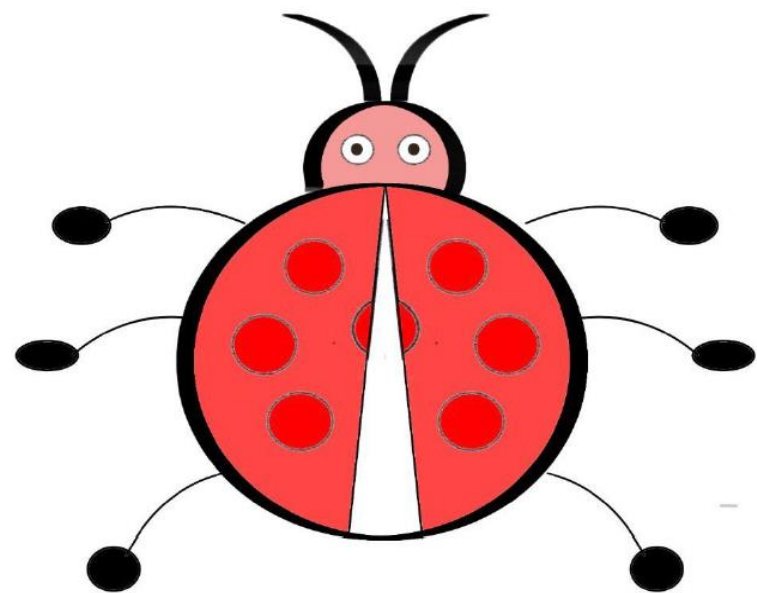
执行测试 t_2 (4, 3, 5),
Mean1输出 2.67, Mean2输出4
执行测试 t_3 (0, 4, 5),
Mean1输出 3, Mean2输出4.5

```
void Mean1(double X[], int L) {  
    double mean, sum;  
    sum = 0;  
    for (int i = 1; i < L; i++) {  
        sum += X[i];  
    }  
    mean = sum / L;  
    printf("Mean: %f\n", mean);  
}
```

```
void Mean2(double X[], int L) {  
    double mean, sum;  
    sum = 0;  
    for (int i = 1; i < L; i++) {  
        sum += X[i];  
    }  
    mean = sum / L-1;  
    printf("Mean: %f\n", mean);  
}
```

尝试比较分析不同测试集执行结果, 可以看出, 单调性和非单调性依赖测试集的构建。

回顾与讨论



Bug的反向定义

- 不同修复方法带来不同的Bug
- 不同测试确认带来不同的Bug

- Bug的非单调性给程序员的调试和修复带来挑战。
- Bug的定义和修复过程严重依赖测试集的构建。